GPU-Accelerated Conv2D on Raspberry Pi 4

2025.12, Edge-based deep learning

Our Teammates

- Choe Kibin / SID: 202450829

  Parallel processing, pyOpenGL
- URAIMOV JAMOLIDDIN KULDASHALI UGLI / SID: 202460418

  Parallel processing Design Decisions
- RIOS LOPEZ RAUL ALFREDO / SID: 202550969

  make some simple CNN structure

https://github.com/vertman37/rpi_compute

Our Goal

- RPi4 has a GPU. let’s use it for CNN! (No one on earth didn’t yet)

  (while 4 CPUs, it has GPU called Videocore VI too.)
- Start with basic CNN operations (e.g., Conv2D, MaxPool, ReLU)

  (skip the Residual connection, bottleneck)
- Visualization to build intuition for channel pruning

  Understand basic design concepts such as channels, kernels, ..

Our Challenge

- To Implement from scratch

  OpenGL has GPGPU feature, the Compute Shader.

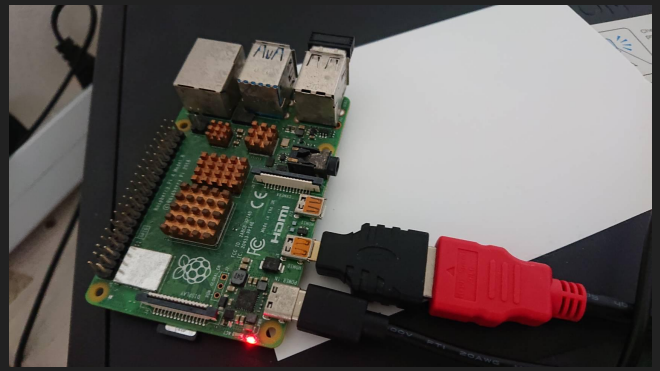
  glsl, shader language is used to implement the operations.
- GPU Architecture for Parallel Processing

  Buffer, L1/L2 Cache, Memory Bandwidth,

  Global Memory Access, shared memory, threads..

  > But It became an opportunity to understand GPU/CNN deeply.

Our Device



Raspberry Pi 4

CPU : 1.5GHz x 2 cycles x 4cores =12GFLOPS

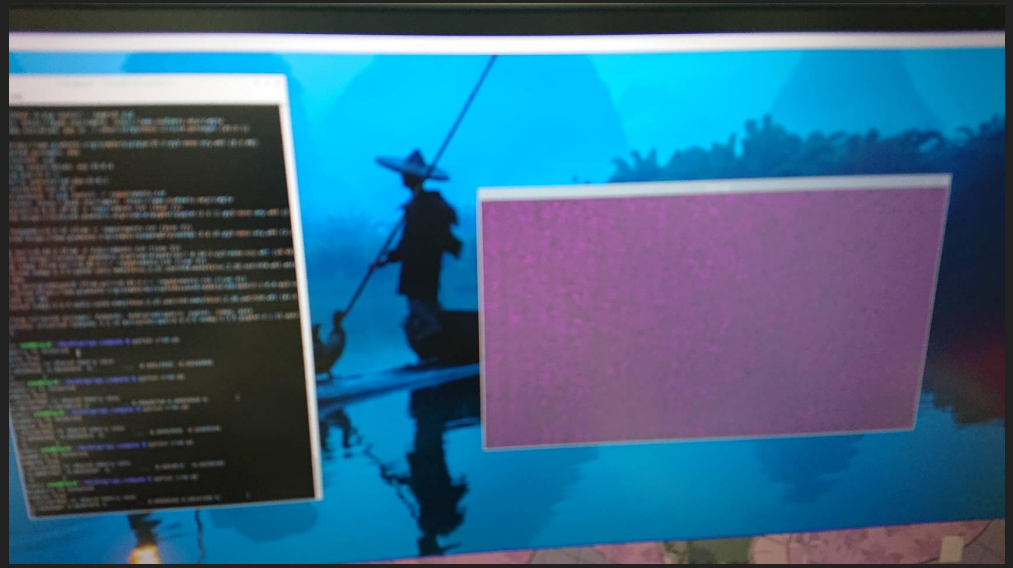
GPU : 0.5GHz x 64 = 32GFLOPS (OpenGL ES3.1)

RAM : LPDDR4 , 12.8GB/s

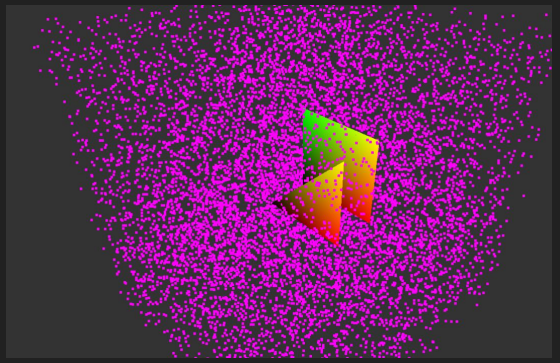
(Most GPU-bound workloads are limited by memory bandwidth)

Hello 3D World

- Basic Particle Movement. (position += speed * dt) (N= ~100k)
- Compute Shader works on RPi 4



Hello Texture



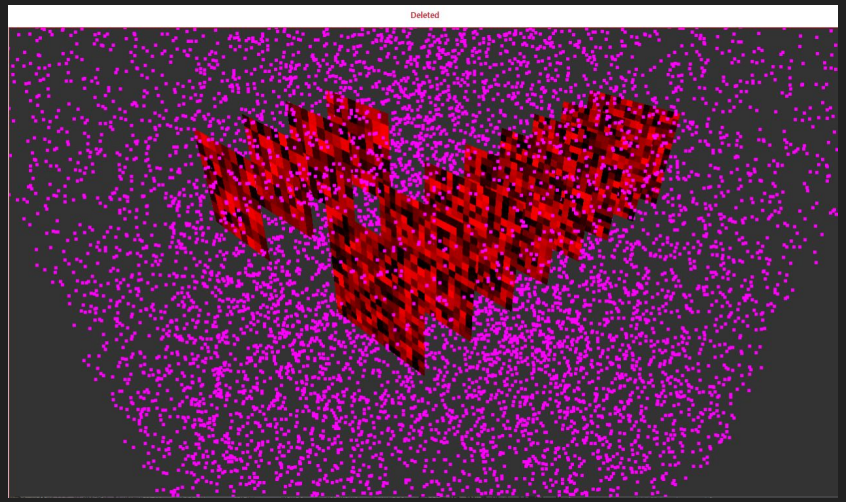
- Texture is.. Image-like 2D data.

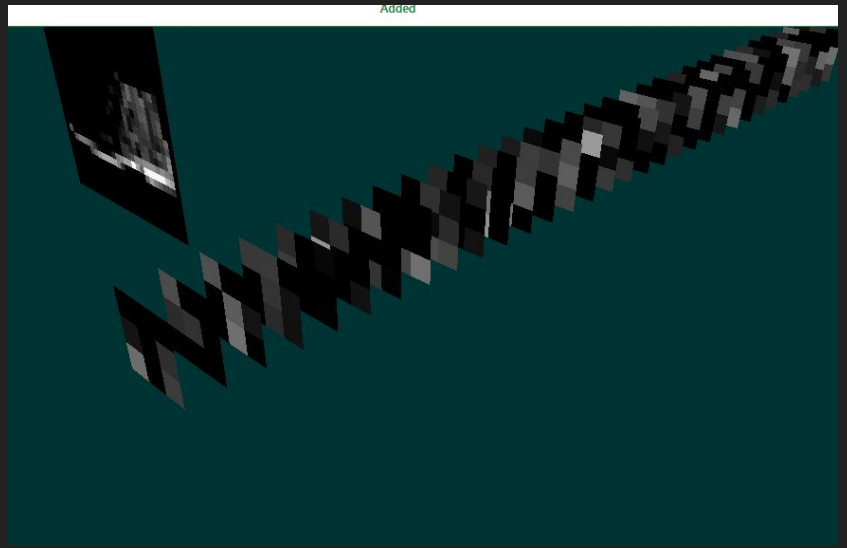
- Texture Array can represent “depth” (or, channel) of an image.
  so we can stack the layers, natively visualize it.

- It has dedicated Texture Cache. fast read speed.

Hello Texture2DArray

- fine-tuned glsl code for access the data.
- now we can see multi-layered data.





Hello CNN

- very basic CNN for Fashion Mnist dataset.
  MNIST is too simple (accuracy 99%), while F-MNIST, 92%.
- Simple Model performs well even without :
  data augmentation, batch normalization..
- CIFAR-10/100 also has Small resolution
  making it a good starting point.

Hello CNN

```python
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)#28x28->14x14
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)#14x14->7x7
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.max_pool2d(x, 2)
        x = F.relu(self.conv2(x))
        x = F.max_pool2d(x, 2)
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)  # dropout for MLP
        x = self.fc2(x)
        return x
```

Export parameters

```python
model = SimpleCNN()
model.load_state_dict(torch.load('fashion_mnist_cnn.pth'))
model.eval()

state = model.state_dict()
for name, param in state.items():
    print(name, param.shape)

w = model.conv1.weight.detach().cpu().numpy()
b = model.conv1.bias.detach().cpu().numpy()
print(type(w), w.shape)
print(type(b), b.shape)
# <class 'numpy.ndarray'> (32, 1, 3, 3)
# <class 'numpy.ndarray'> (32,)
```

```python
# fc1.weight <class 'numpy.ndarray'> (128, 3136)
# fc1.bias <class 'numpy.ndarray'> (128,)
# fc2.weight <class 'numpy.ndarray'> (10, 128)
# fc2.bias <class 'numpy.ndarray'> (10,)

#conv1.weight -> conv1_weight
clean_params = {k.replace('.', '_'): v for k, v :

import numpy as np
np.savez("params.npz", **clean_params)

#finally done to export numpy!
print('saved')
```

fmnist_test_normalized.npz	2025-12-11 오전 1:11	NPZ 파일	30,704KB
params.npz	2025-12-11 오전 12:31	NPZ 파일	1,650KB
save_256	2025-12-11 오후 2:35	PNG 파일	30KB
fashion_mnist_cnn.pth	2025-12-08 오전 11:10	PTH 파일	1,651KB

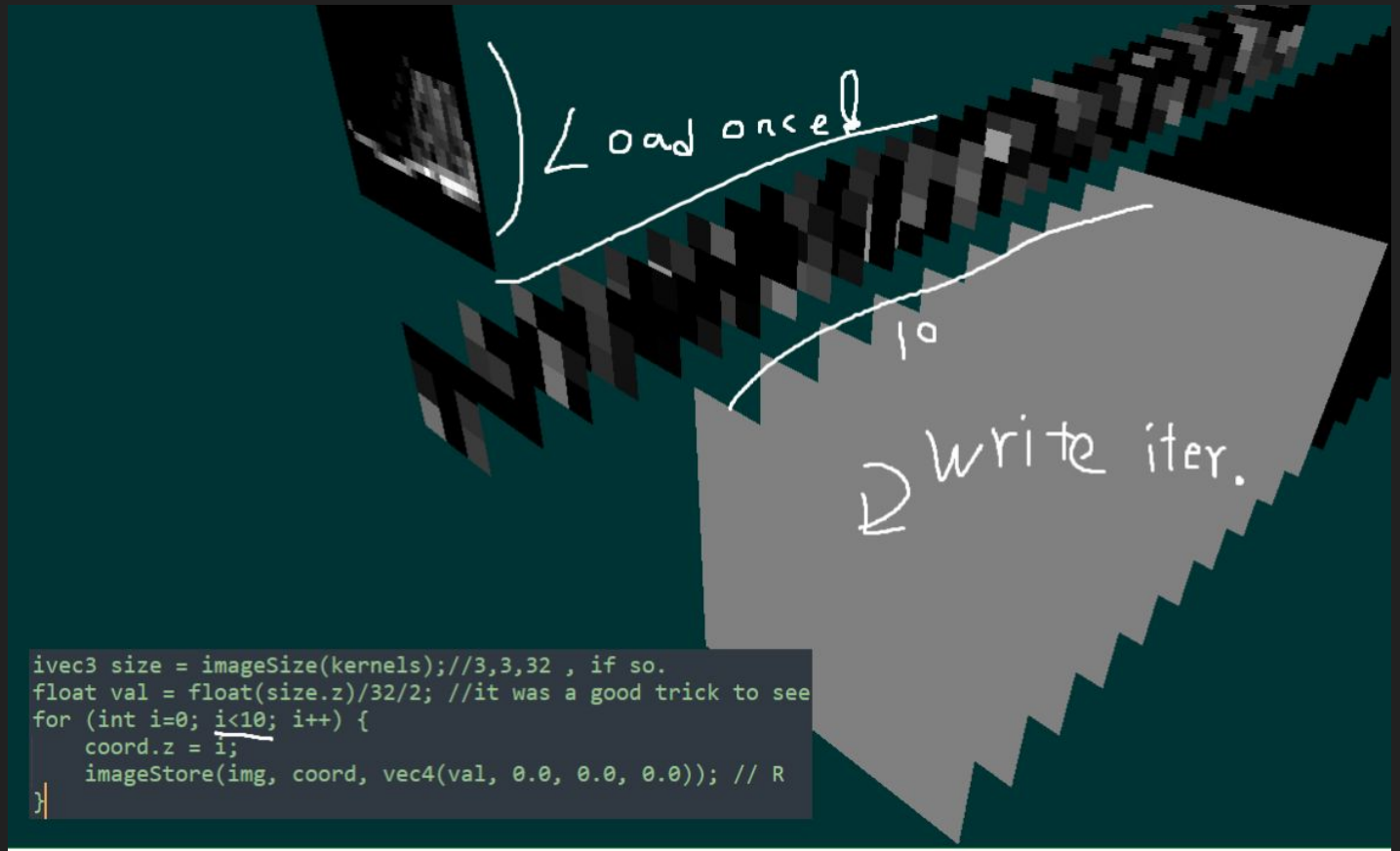
Hello Conv2D

- CNN operations : Conv2d, MaxPool, ReLU

  In GPU computing, memory bandwidth(read/write) is the bottleneck.
- we need to fuse operations

  load once, calculate multiple times, and write once.

Hello Conv2D



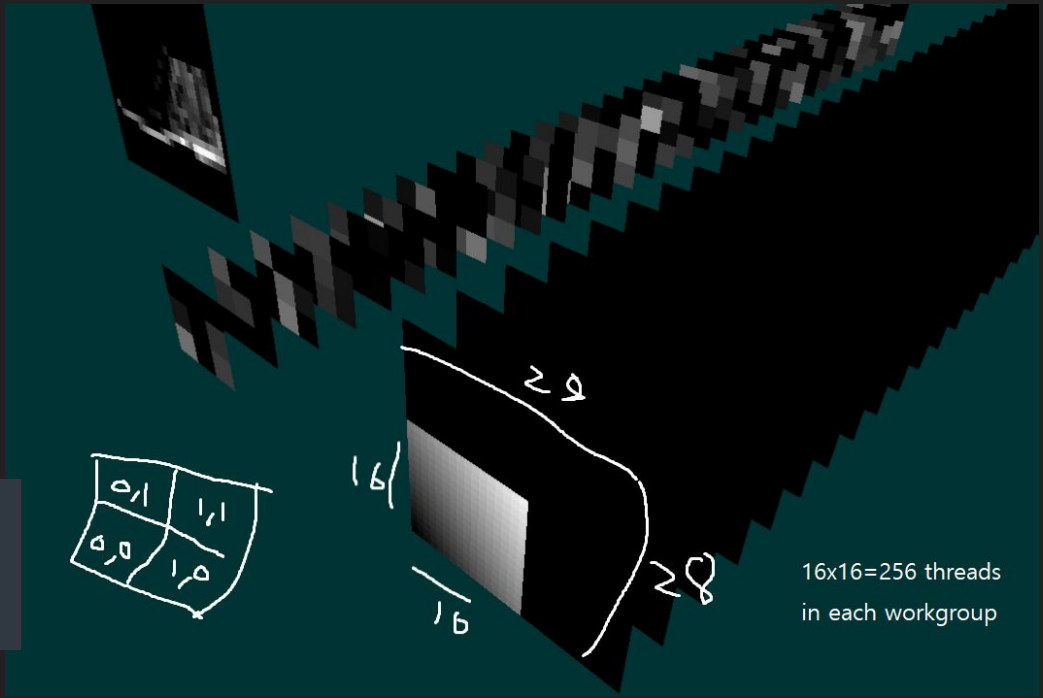

```
ivec3 size = imageSize(kernels);//3,3,32 , if so.
float val = float(size.z)/32/2; //it was a good trick to see
for (int i=0; i<10; i++) {
    coord.z = i;
    imageStore(img, coord, vec4(val, 0.0, 0.0, 0.0)); // R
}
```

threads and workgroup

16x16 =256 threads are in a group

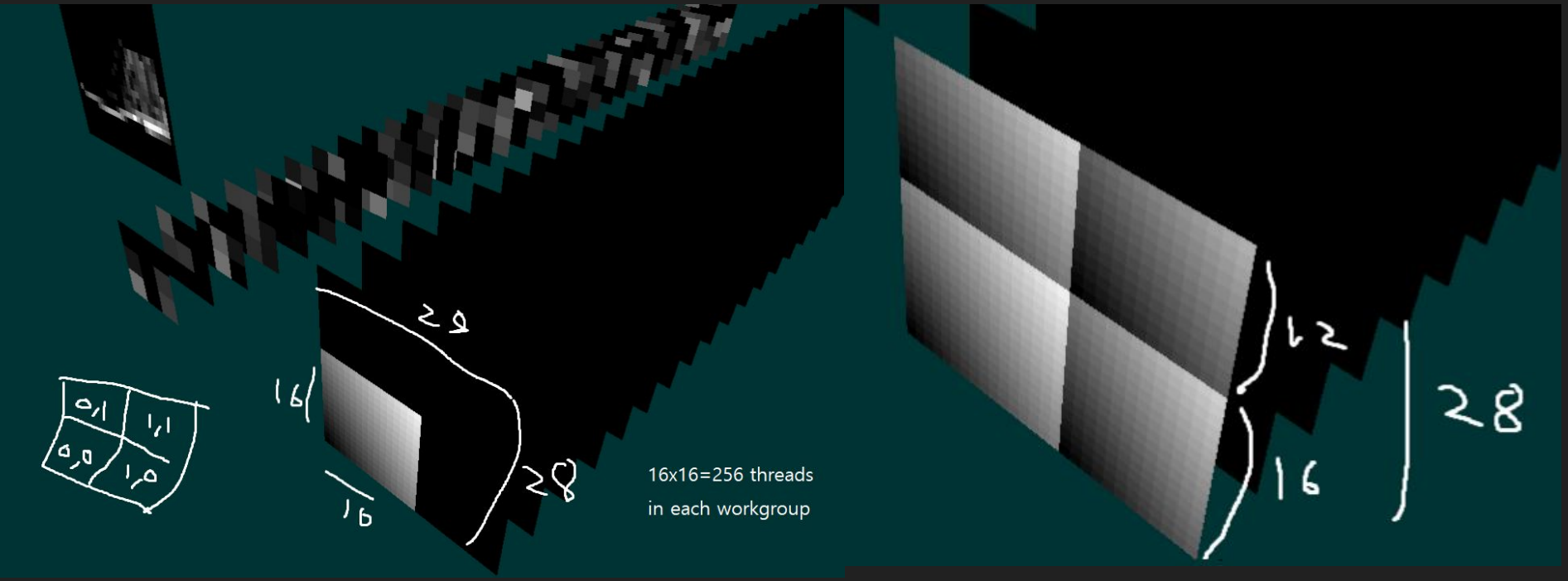
```
compute_src_conv2d ="""
#version 310 es
precision highp float;
precision highp sampler2DArray;
precision highp image2DArray;

uniform float dt;
uniform int Index;

layout(local_size_x = 16, local_size_y = 16) in;
```

```
for ssbo_kernel in conv2d.ssbo_kernels:
    ssbo_kernel.bind(1)
    compute_conv2d.dispatch(2,2,conv2d.out_ch)
```



threads and workgroup

shape = (x=2,y=2, z= out_channels)

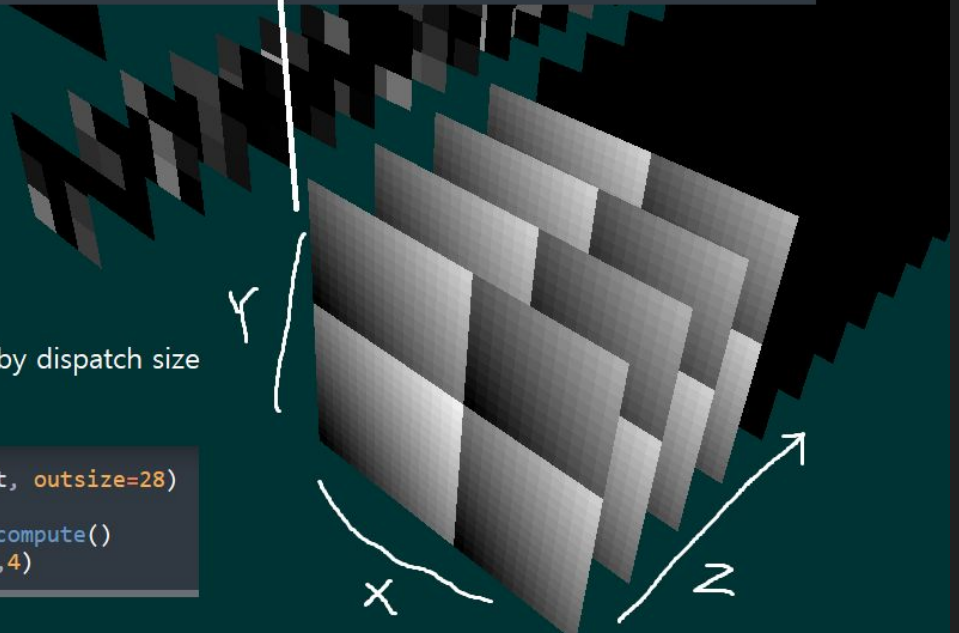


threads and workgroup

more tiled

```
void main() {
    //early discard if coords..
    ivec3 coord = ivec3(gl_GlobalInvocationID.xy, gl_WorkGroupID.z); // layer=0
    float val = float(gl_LocalInvocationID.x)/32.0 + float(gl_LocalInvocationID.y)/32.0;
    imageStore(img, coord, vec4(val, 0.0, 0.0, 0.0)); // R
}
"""
```

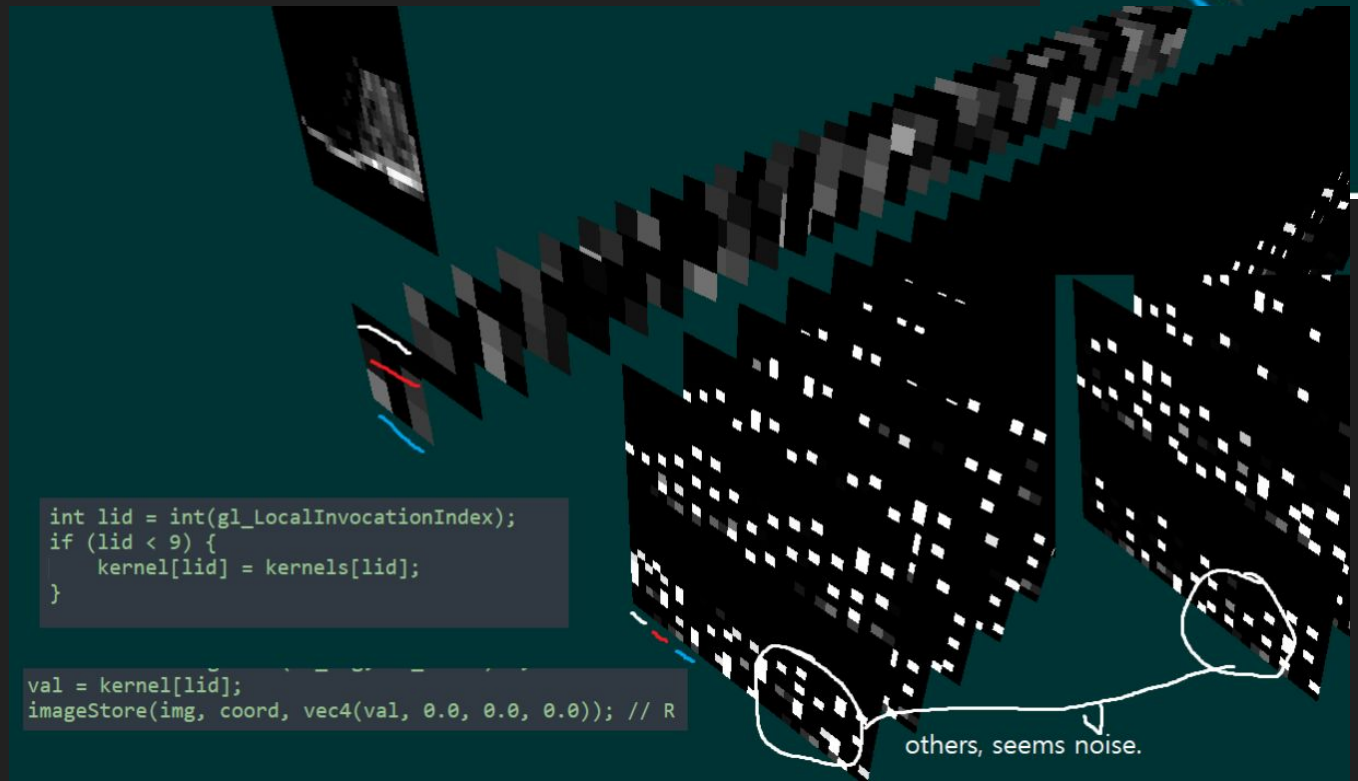
=workgroup.xy * localinvocation.xy (15 + 16*1)



gl_WorkGroupID by dispatch size

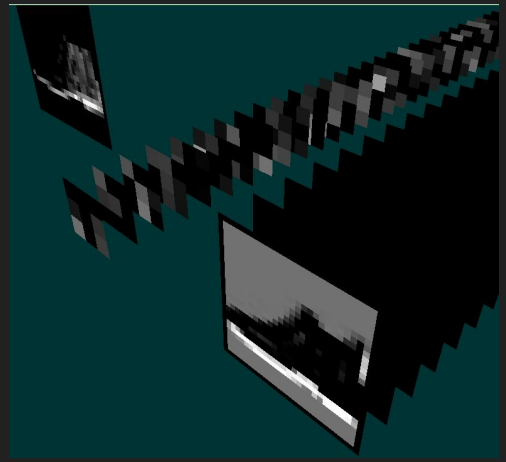
```
conv2d = Conv2d(conv1_weight, outsize=28)

conv2d.tex_outputs[0].bind_compute()
compute_conv2d.dispatch(2,2,4)
```

load kernels using 1D buffer



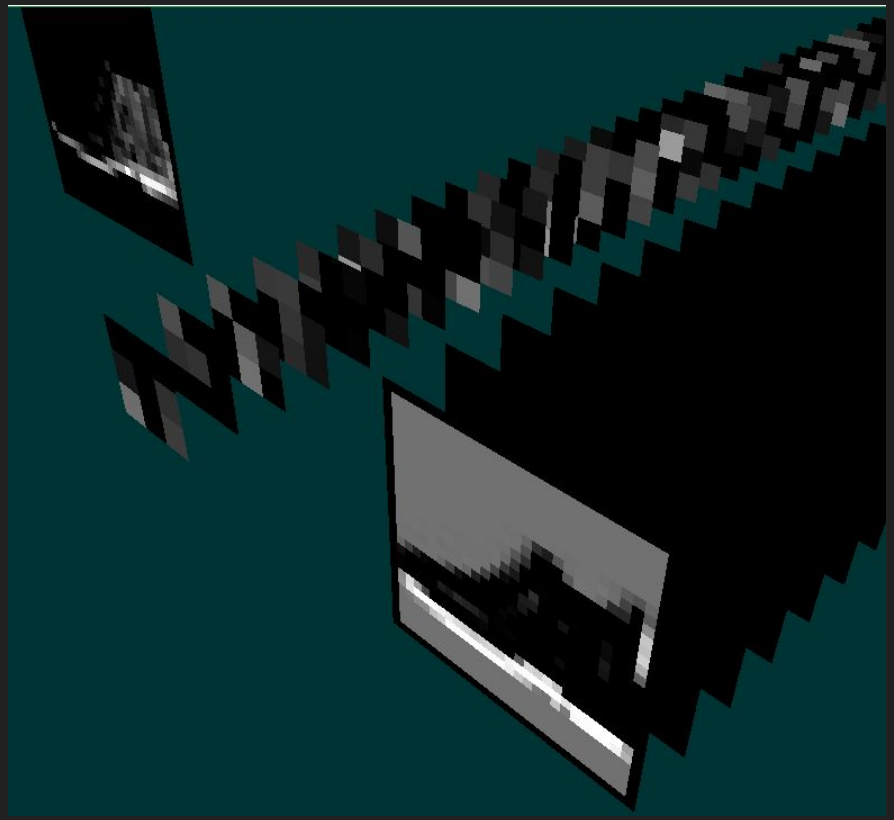

```
int lid = int(gl_LocalInvocationIndex);
if (lid < 9) {
    kernel[lid] = kernels[lid];
}
```

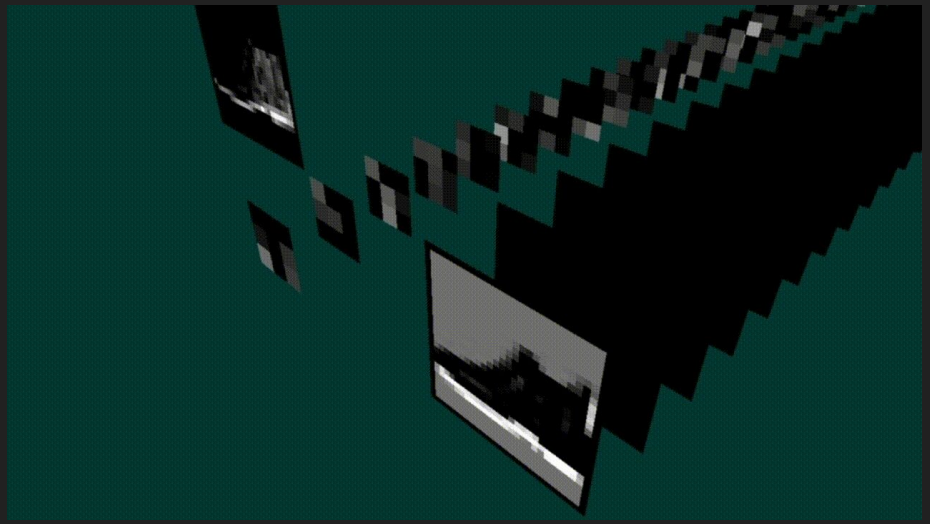
```
val = kernel[lid];
imageStore(img, coord, vec4(val, 0.0, 0.0, 0.0)); // R
```



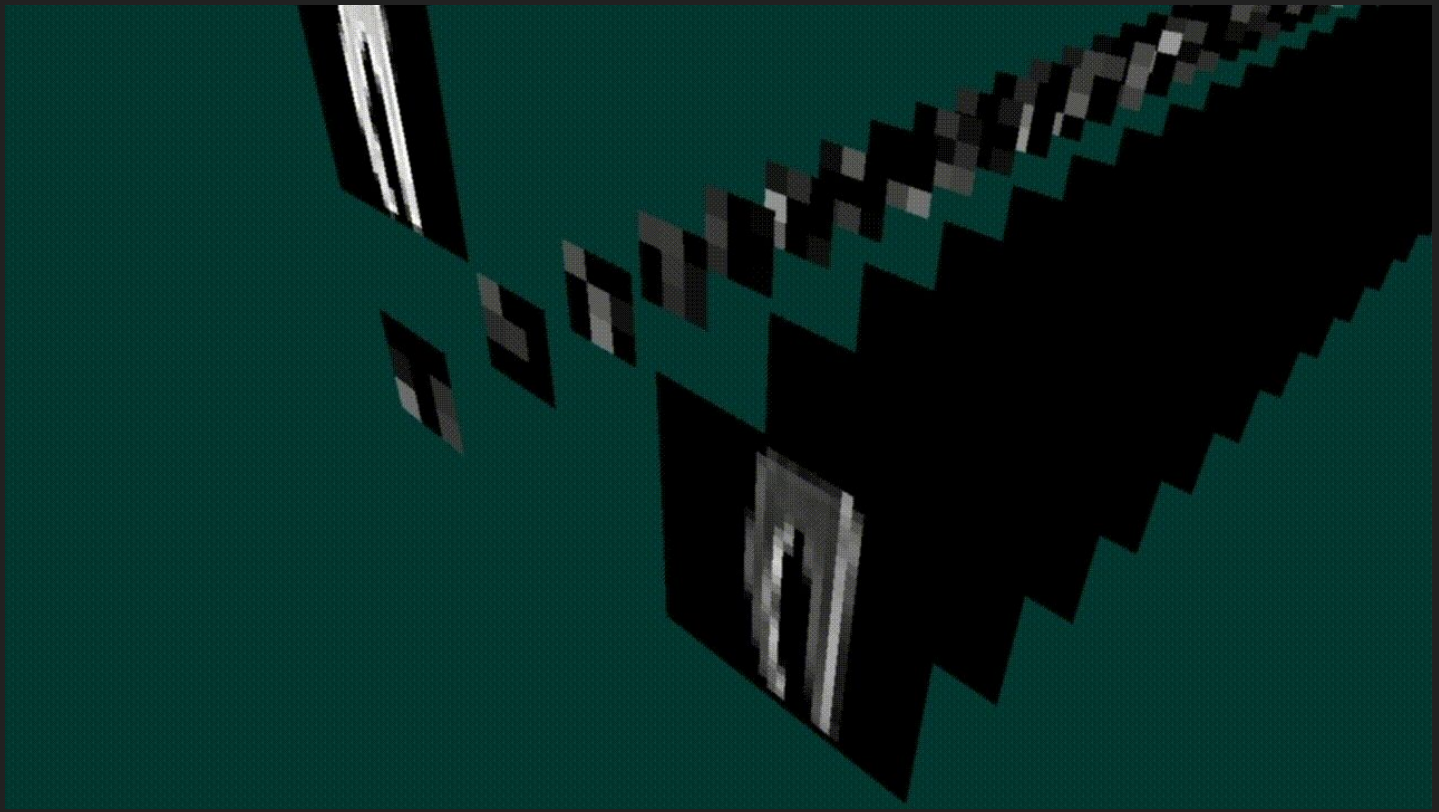
load kernels using 1D buffer - RPi has clean memory

conv operation result





conv operation result



To do & thank you :

- We implemented basic CNN layers, operation.
- implement CNN operations : Conv2d, MaxPool, ReLU
- accelerate it in parallel processing.

- channel pruning (some layers activated or dead)
- small CNN will work great for small images.